SIMILARITY-BASED DUPLICATE RECORD DETECTION USING STRING MATCHING AND MACHINE LEARNING

Allipuram Akhil[1], D. Surya[2], and P. Ruthvik Reddy[3]

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Bowrampet, Hyderabad, Telangana, India

2420030540 · 2420030440 · 2420030546

Faculty Guide: Ms. Bonagani Prathusha, Assistant Professor

Index Terms—Duplicate detection, entity resolution, record linkage, string similarity, blocking, Levenshtein distance, Jaro–Winkler, TF–IDF, machine learning, data quality.

I. INTRODUCTION

Organizations collect customer and entity information through websites, mobile applications, point-of-sale systems, service desks, and legacy databases. When the same person or organization is entered more than once, the resulting records may differ in spelling, punctuation, capitalization, abbreviations, missing values, or contact-field formatting. A simple equality test cannot reliably identify such records. Duplicate entries can fragment customer histories, distort counts, increase storage and maintenance costs, and reduce the quality of downstream analytics.

Duplicate detection, also known as record linkage or entity resolution, determines whether two data records refer to the same real-world entity. The task is challenging because different attributes provide different levels of evidence. A name may contain typographical errors, an email address may be changed, a phone number may be stored with different separators, and an address may use abbreviations or alternate word order. A robust system therefore compares multiple fields and combines the resulting evidence.

This paper presents a similarity-based duplicate record detection system that integrates data preprocessing, candidate generation through blocking, multi-field similarity computation, supervised machine-learning classification, duplicate clustering, and a web-based analytics interface. The implementation is evaluated on a generated customer-record dataset containing 503 rows. The documented data generator creates 350 original records and injects 153 duplicate records, comprising exact and near-duplicate cases. The current execution dashboard reports 605 candidate pairs, 172 flagged duplicate pairs, and 357 representative records after cleaning.

The system compares Logistic Regression, Random Forest, and XGBoost using a common feature representation. The pairwise feature set includes edit-distance, Jaro–Winkler, Jaccard, TF–IDF cosine, and exact-match indicators. The dashboard reports 0.995 accuracy, 1.000 precision, 0.981 recall, and 0.990 F1-score for the displayed run. The confusion matrix provides 130 true negatives, zero false positives, one false negative, and 51 true positives.

The main contributions are: (1) a modular pipeline that reduces unnecessary comparisons through blocking; (2) a multi-field similarity representation for structured customer records; (3) a comparative evaluation of three supervised classifiers; (4) conversion of pairwise predictions into record clusters and a cleaned dataset; and (5) an interactive interface for inspecting matches and downloading outputs.

The remainder of the paper is organized as follows. Section II reviews related work. Section III identifies the research gap and formalizes the problem. Section IV presents objectives and system requirements. Sections V and VI explain the architecture, preprocessing, and candidate generation. Sections VII and VIII describe similarity features and classification. Section IX discusses implementation and interface design. Section X reports results and analysis. Section XI concludes the paper and outlines limitations and future work.

II. RELATED WORK AND LITERATURE SURVEY

Record linkage has a long history in statistics and database research. Fellegi and Sunter introduced a probabilistic framework in which agreement and disagreement patterns across attributes are used to determine whether two records should be linked [1]. Their work established a principled foundation for separating matching, non-matching, and uncertain record pairs. The framework remains influential because it emphasizes that evidence from several fields should be considered jointly rather than treating a single attribute as decisive.

Approximate string comparison is an important component of entity resolution. Levenshtein distance measures the minimum number of insertions, deletions, and substitutions needed to transform one string into another [2]. Jaro similarity was designed for record-linkage applications and is useful for short strings containing transpositions [3]. Winkler's prefix adjustment improves the Jaro measure for names that share an initial sequence [4]. These measures are especially relevant when customer names contain small typing errors or spelling variations.

Token-based similarity provides a complementary view. Jaccard similarity compares the intersection and union of token sets and can tolerate differences in token order. TF–IDF transforms text into weighted vectors, and cosine similarity estimates the angle-based similarity between those vectors [5]. Combining character-level and token-level measures allows a system to detect different types of variation. Christen provides a broad treatment of data matching, indexing, comparison functions, and duplicate detection, highlighting the importance of designing the complete linkage pipeline [6].

Candidate generation is necessary for scalability. Comparing every pair among n records requires $n(n-1)/2$ comparisons, which grows quadratically. Blocking and indexing reduce the candidate space by grouping records that share selected attributes. Christen surveys indexing techniques for scalable record linkage [7], while Papadakis et al. review blocking and filtering methods and their impact on matching quality [8]. Blocking must balance efficiency and recall: overly restrictive blocks may exclude true duplicate pairs before the classifier can evaluate them.

Learning-based duplicate detection treats each candidate pair as a feature vector and predicts whether the pair is a match. Bilenko and Mooney studied adaptive duplicate detection using learnable string similarity measures [9]. Logistic Regression provides an interpretable probabilistic baseline, while Random Forest can capture non-linear interactions between comparison features [10]. XGBoost uses gradient-boosted decision trees and is designed for efficient learning on structured data [11]. Deep-learning approaches such as the work by Mudgal et al. explore learned representations for entity matching [12], and later pretrained language-model approaches extend the field further [13].

The present project uses classical similarity functions and supervised classifiers rather than a large neural matching model. This choice keeps the system inspectable and computationally practical for a student project while providing a modular foundation for future experimentation with transformer-based matching, active learning, and human review.

III. RESEARCH GAP AND PROBLEM FORMULATION

A review of the literature and practical duplicate-detection workflows identifies several recurring gaps. First, exact-match rules are unable to handle near duplicates created by spelling errors, punctuation, capitalization, abbreviations, and inconsistent formatting. Second, using only one similarity function can overlook evidence present in other fields. Third, exhaustive pairwise comparison is expensive as the number of records increases. Fourth, a binary prediction alone is not sufficient for operational use: users need to inspect the two records, understand the similarity evidence, and review uncertain matches before merging data.

A further practical gap exists between model evaluation and data cleaning. Some systems report classification metrics but do not convert predicted pairs into entity groups or produce a representative cleaned dataset. A useful workflow should connect the stages from raw input to reviewable duplicate pairs and a downloadable output. It should also distinguish pair counts from the number of records removed, because one record may participate in several duplicate relationships.

Let the input dataset be $R = \{r1, r2, \ldots, rn\}$. Each record contains structured attributes such as name, email, phone number, and address. The objective is to identify a set of pairs $M = \{(ri, rj)\}$ for which both records refer to the same underlying entity. For each candidate pair, a feature vector x_{ij} is constructed from field-level comparison functions. A classifier $f(x_{ij})$ produces a duplicate or non-duplicate prediction.

The pipeline must satisfy four requirements. The first is data normalization, so that superficial differences do not dominate comparison. The second is candidate generation, which reduces the number of pairs requiring detailed comparison. The third is multi-field evidence aggregation, which considers both exact and approximate similarities. The fourth is record-level consolidation, which transforms predicted pair links into clusters and selects a representative record for each cluster.

The project objectives are: 1) normalize customer attributes and identify incomplete records; 2) generate candidate pairs using a blocking key and a neighboring-name safety mechanism; 3) calculate Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine, and exact-match features; 4) train and compare Logistic Regression, Random Forest, and XGBoost; 5) tune model hyperparameters through grid search and five-fold cross-validation, as documented in the project design; 6) create a duplicate-pair report and a cleaned unique-record dataset; and 7) provide a web interface for dataset upload or generation, result inspection, visualization, and download.

The scope is structured customer-record deduplication. The current evaluation uses generated data and should not be interpreted as proof of performance on every real-world customer database.

IV. SYSTEM REQUIREMENTS AND HIGH-LEVEL ARCHITECTURE

The system is organized as a sequence of loosely coupled processing stages. The input layer accepts a CSV file or generates a synthetic customer dataset. The preprocessing layer standardizes text and phone values. The candidate-generation layer groups likely related records and creates pairs for detailed comparison. The feature layer calculates similarity values for each pair. The model layer predicts duplicate status. The consolidation layer groups linked records and chooses a representative. Finally, the presentation layer displays evaluation results and provides downloadable artifacts.

The architecture separates pair-level matching from record-level cleaning. Pair-level matching answers whether two records are likely to represent the same entity. Record-level consolidation combines the predicted links into connected components. This distinction is important because a single entity can be represented by three or more rows, producing multiple pairwise relationships. The number of flagged pairs therefore does not directly equal the number of removed rows.

The system has the following functional modules: • Dataset ingestion: reads structured records and validates the required columns. • Data preparation: handles missing values, standardizes strings, and normalizes phone numbers. • Blocking: creates candidate groups using normalized name and phone information. • Similarity engine: calculates character, token, vector, and exact-match features. • Model evaluation: trains and evaluates the selected supervised classifiers. • Duplicate clustering: merges connected duplicate relationships and chooses representative records. • Dashboard and export: presents charts, metrics, pair details, and cleaned output files.

Fig. 1. Functional architecture of the similarity-based duplicate detection pipeline.

The documented implementation uses Python for data processing and machine learning. pandas and NumPy support tabular manipulation; scikit-learn supplies preprocessing, vectorization, and model utilities; XGBoost provides the gradient-boosting classifier; Python-Levenshtein and jellyfish provide string-comparison functionality; and matplotlib/seaborn support visual analysis. The web interface is implemented with Flask and provides dataset generation or upload, asynchronous processing, progress updates, pair inspection, charts, and downloads.

Fig. 1 illustrates the processing flow. The dashboard is titled “Duplicate Detection Intelligence Studio” and exposes summary indicators for input records, candidate pairs, flagged pairs, and unique records. It also includes similarity-score distribution, model comparison, confusion-matrix information, and a searchable table of detected pairs.

System requirements include a structured CSV input, Python runtime and project dependencies, sufficient memory to hold the record and pair tables, and a modern browser for the web interface. The current project is designed for local execution and experimental evaluation. Production deployment would require additional controls such as authentication, audit logging, access restrictions, secure storage, and a review workflow for uncertain matches.

V. DATASET DESCRIPTION AND PREPROCESSING

The project uses a generated customer dataset to evaluate duplicate detection under controlled conditions. The documented generator produces 503 rows: 350 original customer records and 153 injected duplicates. The injected cases include 65 exact copies and 88 near duplicates. The generator selects 122 underlying source records for duplication, and the raw dataset contains 18 rows with at least one missing field. These characteristics provide both straightforward exact-match examples and more difficult cases where one or more attributes are altered.

The record schema contains a record identifier, name, email, phone number, address, and duplicate_of relationship information. The duplicate_of field provides a ground-truth relationship for supervised evaluation. It is not treated as an ordinary input feature for prediction; rather, it supports label construction and evaluation. This separation is important to prevent target leakage.

Preprocessing converts missing text values to empty strings for comparison and normalizes the fields. Names, email addresses, and addresses are converted to lowercase, whitespace is collapsed, and unsupported special characters are removed. Phone numbers are standardized by retaining digits only. For example, “999-673-4137” and “9996734137” become equivalent normalized strings. Normalization reduces differences that are caused only by formatting.

Missing values require careful treatment. An empty field should not be considered an exact match merely because both records have no value. Instead, missingness can be represented separately or the relevant comparison can return a neutral score. The project flags rows containing missing fields so that data completeness can be inspected. A future production version should preserve the original raw values alongside normalized values to support auditability and reversible cleaning.

After normalization, the system prepares candidate pairs. The pair table stores identifiers for both records, original attribute values, feature scores, a combined score, the reference duplicate label, and the predicted label. The current supplied candidate_pair_scores.csv contains 605 rows and 15 columns. This file is the central artifact for pair-level model evaluation and duplicate inspection.

The dataset is synthetic, which enables controlled generation of exact and near-duplicate examples. However, generated records may not fully represent the complexity of real data. Real customer datasets can include multilingual names, shared household phone numbers, reused email addresses, inconsistent address structures, and correlated missing values. The synthetic benchmark is therefore appropriate for demonstrating the workflow but should be supplemented with independent real-world evaluation before deployment.

VI. CANDIDATE GENERATION AND SIMILARITY FEATURE ENGINEERING

A naive approach compares every record with every other record. For n = 503 records, exhaustive unordered comparison would require 503 × 502 / 2 = 126,253 pairs. Most of these pairs are unlikely to represent the same entity. Candidate generation reduces the number of detailed comparisons by grouping records that share selected characteristics.

The documented blocking strategy constructs a key from sorted first letters of name tokens together with the first three digits of the normalized phone number. Records within the same block are considered as candidate pairs. A neighboring-name safety mechanism also adds adjacent records after sorting by normalized name. The purpose of the additional mechanism is to recover some likely matches that may fall across a strict blocking boundary. The resulting supplied pair-score artifact contains 605 candidate pairs.

For every candidate pair (A, B), the feature engine compares corresponding attributes. Levenshtein similarity is derived from edit distance. If d(a,b) is the edit distance and L is the maximum string length, a normalized similarity can be expressed as: $Slev(a,b) = 1 - d(a,b) / \max(|a|, |b|)$, with a defined value for two empty strings handled separately. A value closer to 1 indicates greater similarity.

Jaro similarity considers matching characters and transpositions. Jaro–Winkler adds a prefix adjustment and is useful for personal names where the beginning of the string is often stable. Jaccard similarity compares token sets: $J(A,B) = |A \cap B| / |A \cup B|$. It is useful when address or name tokens are reordered. TF–IDF maps text into weighted vectors, and cosine similarity is: $\cos(u,v) = (u \cdot v) / (\|u\| \|v\|)$. The project applies TF–IDF cosine similarity over combined textual fields.

The implemented feature table includes name_levenshtein, name_jaro_winkler, email_levenshtein, address_levenshtein, address_jaro_winkler, name_jaccard, address_jaccard, phone_exact_match, email_exact_match, tfidf_cosine, and combined_score. The exact feature scaling and combined-score weights should be taken from the implementation source when reproducing the experiment; they are not inferred from the dashboard screenshot.

Exact-match indicators complement approximate text measures. A phone_exact_match value of one indicates equality after phone normalization. An email_exact_match value of one indicates equality after email normalization. These binary signals can be strong evidence but are not sufficient alone because contact details may be missing, shared, or changed.

A weighted combined score can be represented generally as: $C(A,B) = \Sigma k\ wk\ sk(A,B)$, where wk are non-negative weights and sk are normalized comparison signals. The actual weights and normalization are implementation-specific. The combined score is used as an additional pair-level feature and is visualized in the dashboard.

VII. MACHINE-LEARNING CLASSIFICATION AND RECORD CONSOLIDATION

After feature construction, each candidate pair is represented by a numerical vector xij. The target label indicates whether the two records refer to the same entity. The system evaluates Logistic Regression, Random Forest, and XGBoost using the same feature representation so that the comparison is based on the same candidate-pair evidence.

Logistic Regression estimates the probability of a duplicate relationship using a sigmoid transformation: $P(y = 1 \mid x) = 1 / (1 + \exp(-(\beta 0 + \beta \blacksquare x)))$. The model is relatively interpretable and provides a useful baseline for structured comparison features. A decision threshold converts the estimated probability into a duplicate or non-duplicate label.

Random Forest constructs an ensemble of decision trees using bootstrapped samples and feature subsets. Individual tree predictions are aggregated to produce the final class estimate. This approach can model non-linear interactions, such as a high name similarity being more persuasive when accompanied by an exact phone match.

XGBoost builds an additive ensemble of decision trees through gradient boosting. Each successive tree is fitted to improve the objective based on the current ensemble. Regularization and tree constraints help control model complexity. XGBoost is often effective for tabular classification, although its performance depends on hyperparameter selection and validation design.

The original project design documents a 70/30 train-test split and grid search with five-fold cross-validation. Hyperparameter tuning should be performed only on the training partition, with the held-out test set reserved for final evaluation. Pair-level splitting must also be designed carefully: if records or entities from the same source appear in both training and test data, leakage can make performance appear higher than it would be on unseen entities. A stronger evaluation should split by underlying entity or source record where possible.

Predicted duplicate pairs are converted into record groups using a union-find data structure. Initially, each record forms its own set. When a pair is predicted as duplicate, the sets containing its two records are joined. At the end, connected components represent candidate entity clusters. For each cluster, the documented implementation retains the record with the fewest missing fields as the representative. The other rows are excluded from the cleaned unique dataset.

This consolidation strategy is practical but has a known risk: a false-positive edge can merge two otherwise distinct entities into one cluster. For that reason, production use should support manual review, confidence thresholds, and a reversible merge log. Ambiguous matches should be presented for review rather than automatically deleted.

VIII. SOFTWARE IMPLEMENTATION AND DASHBOARD

The project is implemented as a modular Python application. The data-processing layer loads the input CSV, validates required columns, and creates normalized comparison fields. The candidate-generation layer applies blocking and neighboring-name logic. The similarity layer calculates string and vector features. The model layer trains the classifiers and produces pair-level predictions. The consolidation layer forms duplicate clusters and exports representative records.

The documented Python stack includes pandas, NumPy, scikit-learn, XGBoost, Python-Levenshtein, jellyfish, matplotlib, and seaborn. pandas and NumPy support data transformation. scikit-learn provides model utilities and TF–IDF vectorization. Python-Levenshtein and jellyfish support edit-distance and Jaro–Winkler comparisons. XGBoost provides the boosted-tree classifier. Visualization libraries are used for charts and model analysis.

The web interface is implemented with Flask. It can generate a synthetic dataset or accept a CSV containing customer name, email, phone, and address fields. The application supports asynchronous execution and progress updates so that the browser can remain responsive while the matching workflow runs. Users can inspect candidate pairs and download the generated outputs.

The output artifacts include duplicate_records_report.csv, cleaned_unique_dataset.csv, model_performance_metrics.json, model-comparison charts, a confusion matrix, a similarity-score distribution, and a dataset-size comparison. The pair report exposes the two record identifiers and the corresponding field values, together with similarity measures such as combined_score, tfidf_cosine, and name_jaro_winkler. This evidence helps a reviewer understand why a pair was flagged.

The current dashboard, “Duplicate Detection Intelligence Studio,” presents a completion message and four summary indicators: input records, candidate pairs, pairs flagged, and unique records. It includes a similarity-score histogram that distinguishes flagged and non-flagged pairs, a dataset-reduction chart, a model evaluation table, a comparative metric chart, a confusion matrix, and a searchable detected-pairs table.

The interface is intended to support inspection rather than hide the model's decisions. The table allows a user to examine names, email addresses, phone numbers, addresses, and similarity scores for each flagged pair. This is useful when two records have similar names but conflicting contact details. The downloadable cleaned dataset makes the workflow useful beyond the dashboard itself.

Security and deployment hardening remain outside the demonstrated experiment. A production deployment should add authentication, role-based access, encrypted transport, input-size limits, validation against malformed CSV files, audit logs, and safeguards against accidental destructive merges.

The supplied dashboard add-on is implemented with Streamlit and Plotly. It supports project-dataset selection or CSV upload, a similarity-threshold control for unlabelled data, searchable duplicate-pair inspection, model comparison, confusion-matrix display, and downloads of duplicate-pair, cleaned-record, and candidate-score CSV files. The add-on imports preprocessing, feature-engineering, and model functions from a separate src directory. Because those source modules were not included in the add-on archive, exact blocking keys, feature weights, split settings, and hyperparameters are not independently verified here.

For uploads without a usable duplicate_of ground-truth field, the application uses the selected combined-similarity threshold and does not report supervised accuracy, precision, or recall. This distinction is important when interpreting results from unlabelled data.

IX. EXPERIMENTAL SETUP AND RESULTS

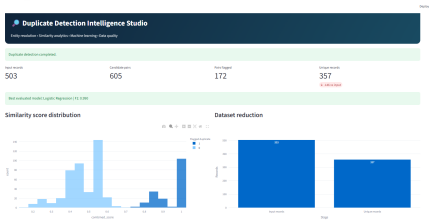


Fig. 1. Duplicate Detection Intelligence Studio dashboard overview.



Fig. 2. Model evaluation comparison for three classifiers.

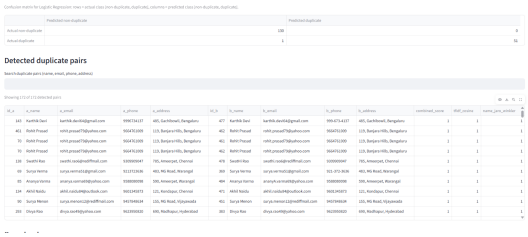


Fig. 3. Confusion matrix and detected duplicate-pair table.

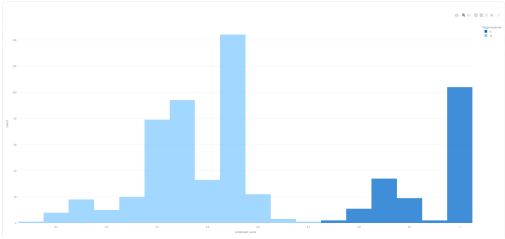


Fig. 4. Distribution of combined similarity scores.

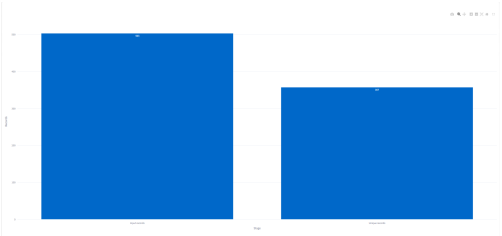


Fig. 5. Dataset reduction from 503 input records to 357 unique records.

The experimental dataset contains 503 input records. The documented generator describes 350 original records and 153 injected duplicates, including 65 exact copies and 88 near duplicates. The generator selects 122 underlying source records for duplication, and 18 rows contain at least one missing field. The current dashboard reports 605 candidate pairs, 172 flagged duplicate pairs, and 357 unique records after consolidation.

The reduction in record count is 503 − 357 = 146 records. Relative to the input size, the reduction is (146 / 503) × 100 = 29.02%, approximately 29.0%. The flagged-pair rate is (172 / 605) × 100 = 28.43%, approximately 28.4%. These percentages measure different things. The first is a record-count reduction; the second is the proportion of candidate pairs classified as duplicates.

The current dashboard reports the same rounded metrics for all three evaluated models: accuracy 0.995, precision 1.000, recall 0.981, and F1-score 0.990. Logistic Regression is identified by the dashboard as the best evaluated model. Since all displayed values are rounded to three decimal places, the screenshot does not establish whether unrounded scores are identical or whether the model selection is based on a difference beyond the displayed precision.

The confusion matrix displayed for Logistic Regression is shown in Table II. It contains 130 true negatives, zero false positives, one false negative, and 51 true positives, for 182 evaluated candidate pairs. Accuracy is (130 + 51) / 182 = 0.9945. Precision is 51 / (51 + 0) = 1.0000. Recall is 51 / (51 + 1) = 0.9808. The harmonic mean of precision and recall is approximately 0.9903. These values round to the dashboard metrics.

The high precision indicates that no false-positive duplicate prediction appears in the displayed evaluation sample. The recall indicates that one duplicate example was missed. In data cleaning, false positives and false negatives have different costs. A false positive can merge distinct people, while a false negative leaves duplicate records in the dataset. A threshold should therefore be selected according to the intended application and the relative impact of each error.

The similarity-score histogram shows a concentration of non-duplicate pairs at lower and middle combined scores and flagged pairs at higher scores, including a visible concentration near 1.0. This separation is consistent with the use of a composite similarity feature. It should not be interpreted as independent proof of correctness for every flagged pair.

The result is promising for the generated evaluation sample. It does not establish generalization to all real customer datasets. Independent labels, leakage-safe splitting, repeated validation, and external datasets are required for stronger evidence.

X. DISCUSSION, LIMITATIONS, AND THREATS TO VALIDITY

The results demonstrate the value of combining several comparison signals. Character-level similarity can detect small spelling differences, token-level similarity can tolerate reordered words, vector similarity can capture overlap across combined text fields, and exact-match indicators can provide strong evidence for normalized phone and email values. The classifier can learn how these signals interact instead of applying one universal rule.

Blocking is an important efficiency component. The theoretical all-pairs count for 503 records is 126,253, while the supplied candidate-pair artifact contains 605 pairs. This is a substantial reduction in the number of detailed comparisons. However, candidate reduction alone is not enough to establish blocking quality. Candidate recall should be measured by determining what proportion of all known duplicate pairs survive the blocking stage. The current screenshots do not provide that measurement.

All three models show the same rounded metrics in the current dashboard. It is therefore appropriate to report similar displayed performance rather than claiming that one classifier is decisively superior. The dashboard selects Logistic Regression as the best evaluated model, but the exact selection rule and unrounded scores should be inspected in the implementation before making a stronger claim.

Several limitations affect the interpretation of the experiment. First, the dataset is generated rather than collected from a production customer database. The corruption patterns may not represent the full range of real-world variation. Second, the exact combined-score weights and classification threshold are implementation-specific and are not visible in the dashboard. Third, the current result screenshots do not show the full split configuration, random seed, or per-model unrounded metrics. Fourth, a pair-level random split can produce leakage if records from the same underlying entity appear across both partitions. Fifth, the confusion matrix covers 182 examples, and the available evidence does not establish how that subset was selected.

There are also operational limitations. A blocking key based on name initials and phone prefixes may miss records when both fields are corrupted. Union-find clustering can propagate a false-positive link through a connected component. Selecting the record with the fewest missing fields does not resolve conflicting values across records. A production system should preserve all source values, provide field-level survivorship rules, and allow a human reviewer to approve or reject merges.

Future evaluation should include a public or institutionally approved real-world dataset, independent human labels, entity-level train-test separation, repeated stratified validation, blocking recall, precision-recall curves, threshold calibration, and confidence intervals. Error analysis should group false positives and false negatives by corruption type, such as name typo, phone formatting, missing email, or address variation. These analyses would identify where the feature representation needs improvement.

The system should be viewed as an experimental decision-support tool. It demonstrates an end-to-end architecture and high reported scores on the supplied synthetic dataset, while further validation is necessary before operational deployment.

A. Entity Resolution as a Pairwise Decision Problem

Entity resolution is difficult because the same attribute can have different reliability across records. A name is often informative but may contain spelling errors or shortened forms. Email addresses can be highly discriminative but may be shared or changed. Phone numbers can be reformatted or reassigned. Addresses may contain abbreviations, apartment details, or reordered locality components. A robust system therefore treats each field as a source of evidence and avoids assuming that one attribute is universally decisive. Pairwise comparison provides a practical formulation: the system considers two records at a time, calculates evidence for their

correspondence, and estimates whether they refer to the same entity.

B. Candidate Generation and Computational Cost

Candidate generation is a filtering stage that precedes expensive similarity calculations. If all n records are compared against every other record, the number of unordered pairs is n(n−1)/2. This becomes impractical as n grows. Blocking assigns records to groups based on selected keys and compares only records that share a group. A useful blocking strategy should reduce the candidate count while retaining as many true duplicate pairs as possible. The dashboard source confirms blocking and adjacent-name candidates, but the exact key construction is implemented in the separate feature-engineering module that was not part of the uploaded add-on. Consequently, this paper reports the observed 605 candidate pairs without claiming an unverified blocking formula.

C. Blocking Recall and Pair Completeness

The effectiveness of candidate generation should be evaluated separately from the classifier. Pair completeness, also called blocking recall, is the proportion of all known true duplicate pairs that appear in the candidate set. Pair quality is the proportion of candidate pairs that are true matches. A highly selective blocking rule can produce a small candidate set with good pair quality but may miss true matches. Conversely, a permissive rule can retain more true matches while increasing the classifier workload. Both measures should be reported in future experiments. The available dashboard reports candidate-pair count but does not expose a separate blocking-recall result.

D. Character-Level Similarity

Levenshtein distance is appropriate for short strings affected by insertions, deletions, and substitutions. For example, a one-character spelling difference between two names creates a small edit distance relative to their length. A normalized score maps the distance to a range that is easier to combine with other features. Jaro–Winkler similarity is also suited to short strings and gives additional weight to common prefixes. This can be useful for names where the initial characters are often retained even when the remainder contains a typo. However, prefix similarity can be misleading when many people share common first-name prefixes, so it should be interpreted with other fields.

E. Token and Vector Similarity

Token-based similarity reduces sensitivity to word order. An address such as “Road 12, Central Area” may be represented with the same tokens as “Central Area, Road 12”. Jaccard similarity measures the proportion of shared tokens across the union of token sets. TF–IDF cosine similarity adds term weighting: common words contribute less discriminative information than terms that are relatively distinctive in the compared text collection. The candidate-pair artifact contains both Jaccard features and a TF–IDF cosine score, indicating that the implementation combines token overlap with vector-space evidence. The precise text concatenation and vectorizer settings are defined in the omitted source module and should be documented from that code before publication.

F. Exact-Match Indicators and Missing Values

Exact-match indicators provide strong but narrow evidence. A phone_exact_match feature can detect equality after the phone values have been normalized, and email_exact_match can identify matching email strings. These signals are useful when values are present and stable. They should not be treated as proof of identity because shared contact details, recycled phone numbers, family email accounts, and data-entry errors can occur. Missing values require special care. Two empty strings should not automatically be considered a match, since the absence of information is not positive evidence that two records refer to the same person. The project dashboard reports missing-field rows and should be used to review the completeness of the input dataset.

G. Composite Similarity and Decision Thresholds

The combined similarity score summarizes several comparison signals into a single value. It supports visualization and provides a threshold-based decision path when labelled training examples are unavailable. The threshold slider in the dashboard is explicitly described as applying to unlabelled data. Raising the threshold generally makes the system more conservative, potentially reducing false positives while increasing false negatives. Lowering the threshold can recover more possible matches but may increase incorrect merges. A production threshold should be selected using a validation set and the relative costs of the two error types. Since the exact weights used to construct the combined score are contained in the missing feature-engineering module, this paper does not assign numerical weights.

H. Evaluation Metrics

Accuracy is the proportion of all evaluated pairs classified correctly. Precision measures the proportion of predicted duplicates that are actual duplicates, while recall measures the proportion of actual duplicates recovered by the classifier. F1-score is the harmonic mean of precision and recall. These measures provide complementary information. Accuracy can be misleading when the classes are imbalanced, because a model may achieve high accuracy by predicting the majority class. Precision and recall are particularly important in duplicate detection because false-positive merges and unresolved duplicates have different consequences. Reporting the confusion matrix alongside scalar metrics makes the types and counts of errors explicit.

I. Interpretation of the Confusion Matrix

For the displayed Logistic Regression matrix, the true-negative count is 130, the false-positive count is zero, the false-negative count is one, and the true-positive count is 51. The total number of evaluated examples is 182. The matrix therefore indicates that 51 of 52 actual duplicate examples were detected in this evaluation subset, while all 130 non-duplicate examples were classified correctly. These values produce a recall of approximately 98.08 percent and a precision of 100 percent. The result should be interpreted as a test-subset result rather than a guarantee that no false positives will occur on future data.

J. Comparison of the Three Classifiers

Logistic Regression, Random Forest, and XGBoost provide three different learning approaches for the same feature representation. Logistic Regression offers a relatively simple decision boundary and can be easier to interpret. Random Forest can model non-linear feature interactions without requiring a linear relationship between each feature and the outcome. XGBoost builds boosted decision trees and can capture complex patterns in structured data. The dashboard reports the same rounded metrics for all three models. This may reflect similar performance at the displayed precision; it does not prove that the models have identical unrounded predictions. The dashboard selects Logistic Regression as the best evaluated model, and the exact tie-breaking logic should be confirmed in the main source module.

K. Record-Level Consolidation

Pairwise predictions must be translated into groups of records. A connected-component strategy links records through predicted duplicate edges. If A is linked to B and B is linked to C, all three records are placed in the same component even if the classifier did not separately predict the A–C pair. This is useful when one entity appears multiple times. The dashboard's union-find implementation makes these joins efficiently. It then calculates a missing-field count and selects a representative record from each component. This strategy is simple and reproducible, but it assumes that pairwise edges are sufficiently reliable. One incorrect edge can merge two otherwise distinct groups, so review and rollback mechanisms are important.

L. Representative Record Selection

Selecting one row from each cluster is a data-survivorship decision. The current dashboard sorts records by their number of missing values and retains the first row within each cluster. This favors records with more complete fields. It does not necessarily choose the newest record, the most authoritative source, or the record with the most accurate contact information. A more advanced implementation could apply field-specific rules, such as preferring a verified email over an unverified email, retaining the most recently updated phone number, or combining non-conflicting attributes after human approval. Any merged record should preserve references to its source rows.

M. Dashboard Interaction and Reproducibility

The Streamlit interface supports two data sources: the bundled project dataset and a user-uploaded CSV. The upload path checks the required schema and unique numeric identifiers before processing. The sidebar exposes a similarity threshold for unlabelled data. The dashboard then displays summary metrics, similarity distribution, dataset reduction, model evaluation, confusion matrix, and a searchable duplicate-pair table. Download controls expose the duplicate report, cleaned dataset, and candidate scores. These artifacts support reproducibility because a reviewer can inspect the exact candidate and output tables rather than relying only on screenshots.

N. Data Validation and Error Handling

Input validation is necessary because malformed records can cause incorrect matches or runtime failures. The dashboard checks for required columns and rejects repeated record identifiers. It converts identifiers to integers and casts text fields to strings after filling missing values. Additional validation would be valuable for production use: checking maximum field lengths, detecting invalid email formats, normalizing country-specific phone numbers, identifying impossible dates if dates are added, and limiting file size. The application should also record a processing summary and retain the original uploaded file so that the cleaning process can be audited.

O. Privacy and Responsible Use

Customer records can contain personal information. A deployed system should restrict access to authorized users, protect data in transit and at rest, avoid logging full contact details unnecessarily, and define a retention policy for uploaded files and generated reports. Duplicate detection should not automatically delete source records without review, especially when false-positive merges could affect customer service, billing, or legal records. The dashboard itself warns that a flagged pair is a candidate relationship and

recommends review of borderline matches. This is an appropriate operational safeguard and should be retained in future versions.

P. Threats to Validity

The first threat is synthetic-data bias: generated names, addresses, and corruption patterns may be simpler than real-world data. The second is possible entity leakage if records associated with the same underlying person are split between training and test partitions. The third is candidate-selection bias because the classifier only sees pairs that survive blocking. The fourth is metric uncertainty because the displayed values are rounded and the available screenshot does not expose the full split protocol. The fifth is implementation reproducibility: the add-on references source modules not included in the supplied ZIP. These limitations do not invalidate the demonstration, but they constrain the strength of claims that can be made from the current results.

Q. Recommended Extended Evaluation

A stronger evaluation should use multiple random seeds and repeated stratified splits, while ensuring that all records belonging to the same underlying entity remain in a single partition. A held-out external dataset should be used to test generalization. The evaluation should report precision, recall, F1-score, specificity, area under the precision-recall curve, and confidence intervals. Blocking recall should be measured before model classification. Threshold calibration should be performed on a validation partition rather than the test set. Error analysis should include examples of false-positive and false-negative pairs and group them by the type of variation that caused the error.

R. Practical Deployment Considerations

For operational deployment, the pipeline should be exposed through a controlled service or internal application with authentication, authorization, audit logging, and secure file handling. Large datasets may require batch processing, database-backed candidate storage, and incremental matching rather than loading every record into memory. Human reviewers should be able to accept, reject, or defer candidate pairs. Accepted merges should be reversible, and the system should store the source record identifiers, model version, feature values, decision threshold, and reviewer action. These capabilities would make the project more suitable for real enterprise data-quality workflows.

S. Error Analysis by Attribute

An effective duplicate-detection evaluation should inspect errors at the attribute level. A false negative may occur when a person's name changes substantially and the contact fields are also incomplete. A false positive may occur when two different people share a common name and address, or when an email or phone value is reused. Grouping errors by the affected fields can reveal whether the feature set relies too heavily on one signal. The current dashboard exposes selected similarity values in the duplicate-pair table, which provides a starting point for this analysis. A future version should also export false-positive and false-negative examples with their ground-truth labels.

T. Human Review Workflow

A human review workflow can reduce the risk of irreversible automated merges. Candidate pairs can be divided into high-confidence, low-confidence, and uncertain groups. High-confidence pairs may be prioritized for review, while borderline pairs can be placed in a manual queue. Reviewers should see the original values, normalized values, field-level similarity features, combined score, and model decision. Their decisions should be logged with reviewer identity, timestamp, and reason. The project dashboard already provides a searchable pair table and cautions users to review borderline pairs; a dedicated approval and rejection workflow is a natural next step.

U. Reproducible Experiment Configuration

Reproducibility requires recording the dataset version, preprocessing configuration, candidate-generation settings, feature definitions, model hyperparameters, random seed, split strategy, and software package versions. The current dashboard exports candidate scores and model metrics, but the uploaded add-on does not include the underlying source modules or a complete experiment manifest. A future experiment should generate a JSON metadata file alongside the CSV outputs. This would allow another researcher to reproduce the candidate set and classifier evaluation and to identify whether changes in results arise from data, features, model configuration, or threshold selection.

V. Interpreting Record Reduction

The reduction from 503 input rows to 357 representative rows is a record-level outcome of clustering and representative selection. It should not be interpreted as a direct estimate of the number of true duplicate entities unless the ground-truth cluster structure is independently verified. The pair report contains 172 predicted duplicate links, but links can overlap within connected components. A cluster of three records may create more than one pair link while reducing the row count by only two. Reporting both pair-level and record-level counts, as the dashboard does, avoids this common ambiguity.

W. Scalability Considerations

For larger datasets, the main computational challenge is candidate generation and pair-feature construction. The quadratic all-pairs baseline becomes expensive even before machine-learning inference begins. Blocking, sorted-neighborhood methods, and inverted-index retrieval can reduce comparisons. Large deployments may also require chunked CSV ingestion, database-backed candidate storage, vectorization reuse, and parallel feature computation. The current project demonstrates candidate reduction on 503 records; performance and memory behavior on much larger datasets remain to be benchmarked.

X. Model Monitoring and Drift

Duplicate patterns can change over time as customer behavior, data-entry interfaces, and source systems evolve. A model trained on one dataset may become less reliable when new abbreviations, phone formats, or address conventions appear. Monitoring should track the distribution of similarity features, the fraction of pairs flagged, the number of uncertain pairs, and reviewer disagreement. Periodic relabelling and retraining can help maintain performance. The current dashboard provides score distributions and summary counts that could form the basis of a monitoring view, although no continuous monitoring service is demonstrated in the supplied add-on.

Y. Limitations of Synthetic Evaluation

Synthetic data makes it possible to create controlled duplicate and non-duplicate examples, but it can introduce regularities that make classification easier than it would be in real environments. If duplicate records are generated by copying a source row and applying a limited set of transformations, a model may learn those transformation patterns. Real-world records may contain correlated errors, multilingual text, shared household details, organization names, and source-specific conventions. The reported metrics should therefore be described as results on the current generated evaluation sample, not as a universal expected accuracy.

Z. Summary of Practical Findings

The practical finding is that the project connects approximate matching with an inspectable data-cleaning interface. The candidate-pair CSV preserves feature-level evidence, the duplicate report supports human inspection, and the cleaned dataset provides a downstream artifact. The displayed evaluation suggests that the three classifiers perform strongly on the current test subset. The most important next step is not simply adding another model; it is strengthening the experiment with leakage-safe splitting, external data, blocking-recall measurement, and a reviewable merge process.

Algorithm 1: Candidate-Pair Classification and Consolidation

Input: customer-record table R, optional ground-truth field, and threshold τ. Output: candidate-pair scores P, predicted duplicate report D, and representative dataset U. Step 1: validate that record_id, name, email, phone, and address exist; reject duplicate or non-numeric identifiers. Step 2: prepare the records through the preprocessing module and retain the source attributes for display. Step 3: generate candidate pairs using the project's blocking and adjacent-name candidate procedure. Step 4: compute the pairwise feature vector for every candidate, including edit-distance, Jaro–Winkler, Jaccard, exact phone/email, TF–IDF cosine, and combined-score features. Step 5: if sufficient ground-truth labels exist, label pairs, split the data, train and evaluate the supported classifiers, select the best evaluated model, and predict candidate-pair labels; otherwise, apply the user-selected threshold τ to the combined score. Step 6: create the duplicate report from pairs predicted as matches and join the original record attributes for inspection. Step 7: initialize one disjoint set per record and union the endpoints of every predicted duplicate pair. Step 8: compute each record's missing-field count and select a representative record for each connected component. Step 9: display summary metrics, charts, evaluation results, searchable pairs, and CSV download controls. This procedure describes the observable orchestration in the supplied dashboard source; the internal details of imported preprocessing, feature, and model functions remain defined by the main project source.

Algorithmic Complexity

Let n denote the number of input records and m the number of generated candidate pairs. Exhaustive pair generation has $O(n^2)$ pair count. The candidate-generation stage aims to produce m far below $n(n-1)/2$. If each candidate pair requires a fixed number of string comparisons over bounded field lengths, pair-feature construction is approximately $O(m \cdot L)$, where L summarizes the average string-comparison cost. The union-find consolidation stage performs m union operations over n records and has near-linear amortized behavior with path compression and union by rank. The practical runtime is therefore strongly influenced by candidate generation and the cost of string/vector feature construction. The current experiment demonstrates 605 candidate pairs from 503 records, but does not provide a formal runtime benchmark.

Output Traceability

A useful property of the implementation is that the candidate-pair score table, duplicate report, and cleaned dataset are separate outputs. The candidate-pair table retains both labels and predicted decisions, making it possible to compare model outcomes against the supplied ground truth. The duplicate report focuses on pairs predicted as duplicates and includes human-readable field values. The cleaned dataset contains one representative row per resulting cluster. Keeping these outputs separate supports traceability and helps reviewers distinguish model evaluation from the final record-consolidation result.

XI. CONCLUSION, FUTURE WORK, AND REFERENCES

This paper presented a similarity-based duplicate record detection system for structured customer information. The workflow combines normalization, blocking, multi-field similarity feature construction, supervised classification, duplicate clustering, and an interactive dashboard. The feature set includes Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine, and exact phone/email match signals. Logistic Regression, Random Forest, and XGBoost are evaluated using a common pair-level representation.

In the documented execution, 503 input records produced 605 candidate pairs. The dashboard flagged 172 pairs and reported 357 representative records after cleaning, corresponding to a reduction of 146 rows or approximately 29.0%. The current dashboard reports 0.995 accuracy, 1.000 precision, 0.981 recall, and 0.990 F1-score. The displayed confusion matrix contains 130 true negatives, zero false positives, one false negative, and 51 true positives. These values indicate strong performance on the displayed evaluation sample, but they should be interpreted in the context of the generated dataset and the available validation evidence.

Future work will focus on evaluating larger real-world datasets, measuring blocking recall, improving candidate generation, learning feature weights, calibrating decision thresholds, and introducing active-learning-based human review. Additional models, including transformer-based entity matching, can be evaluated as extensions. The consolidation stage can be improved with field-level survivorship rules, reversible merge operations, and audit logs. Incremental matching can also be explored so that new records are compared against existing entities without rerunning the entire dataset.

The system provides a practical foundation for structured-record deduplication and demonstrates how approximate string matching and machine learning can be combined in a usable data-quality workflow.

ACKNOWLEDGMENT The authors thank Ms. Bonagani Prathusha, Assistant Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, for her guidance and academic support. The authors also acknowledge the department and institution for providing the academic environment for this project.

REFERENCES [1] I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” Journal of the American Statistical Association, vol. 64, no. 328, pp. 1183–1210, 1969. [2] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, pp. 707–710, 1966. [3] M. A. Jaro, “Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida,” Journal of the American Statistical Association, vol. 84, no. 406, pp. 414–420, 1989. [4] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in Proc. Section on Survey Research Methods, American Statistical Association, 1990, pp. 354–359. [5] G. Salton and M. J. McGill, Introduction to Modern Information Retrieval. New York, NY, USA: McGraw-Hill, 1983. [6] P. Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Berlin, Germany: Springer, 2012. [7] P. Christen, “A survey of indexing techniques for scalable record linkage and deduplication,” IEEE Transactions on Knowledge and Data Engineering, vol. 24, no. 9, pp. 1537–1555, 2012. [8] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, “Blocking and filtering techniques for entity resolution: A survey,” ACM Computing Surveys, vol. 53, no. 2, 2020. [9] M. Bilenko and R. J. Mooney, “Adaptive duplicate detection using learnable string similarity measures,” in Proc. ACM SIGKDD, pp. 39–48, 2003. [10] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001. [11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, pp. 785–794, 2016. [12] S. Mudgal et al., “Deep learning for entity matching: A design space exploration,” in Proc. ACM SIGMOD, pp. 19–34, 2018. [13] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, “Deep entity matching with pre-trained language models,” arXiv:2004.00584, 2020. [14] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, “A comparison of string distance metrics for name-matching tasks,” in Proc. IJCAI, pp. 73–78, 2003. [15] A. Arasu, V. Ganti, and R. Kaushik, “Efficient exact set-similarity joins,” in Proc. VLDB, 2006.